

hw1

Ian Chi

January 26, 2026

1

```
epinions=# SELECT COUNT(*) FROM edges;
count
-----
508837
(1 row)
Time: 26.229 ms
```

2

- 1 is not. the query relies on checking every element in the database even if the active domain of E is small.
 - the only values that ever get returned are x and z that show up in R or S. so this query is DI
 - the formula is only more restrictive than 2 so it is also DI
2. 2 is a theta join over y then project on x and z.

$$\pi_{x,z}(R \bowtie_{R.y=S.y} S) \quad (1)$$

3 is a theta join over y, then a filter with $z = x$, project on x and z.

$$\pi_{x,z}(\sigma_{z=x}(R \bowtie_{R.y=S.y} S)) \quad (2)$$

1.

```
SELECT R.x, S.z
FROM R
JOIN S ON R.y = S.y;
```

2.

```

SELECT R.x, S.z
FROM R
JOIN S ON R.y = S.y
WHERE R.x = S.z;

```

TIMINGS:

```

epinions=# SELECT COUNT(*) FROM (
epinions(#      SELECT DISTINCT R.src AS x, S.dst AS z
epinions(#      FROM edges AS R
epinions(#      JOIN edges AS S
epinions(#      ON R.dst = S.src
epinions(# ) AS full_query;
count
-----
20580980
(1 row)

```

Time: 11004.263 ms (00:11.004)

```

epinions=# SELECT COUNT(*) FROM (
epinions(#      SELECT DISTINCT R.src AS x, S.dst AS z
epinions(#      FROM edges AS R
epinions(#      JOIN edges AS S
epinions(#      ON R.dst = S.src
epinions(#      WHERE S.dst = R.src
epinions(#      ) AS full_query;
count
-----
31100
(1 row)

```

Time: 221.799 ms

3. if you executed them in the order written in RA, then it seems like 2 would be faster, but in fact 3 is faster. this is probably because SQL knew to swap the order so the equivalence of x and z was checked first making the query faster.

3

1. consider the relations G' describing graph G where the schema is source and terminal nodes for edges. (G can be directed).

the k clique problem would be

$$\exists x_1 \dots \exists x_k \bigwedge_{\text{over all combinations of } x_i, x'_i} G'(x_i, x'_i) \quad (3)$$

note that this conversion takes approx n^2 time to complete. $\binom{n}{2} \approx n^2$ asymptotically

this shows this problem is np hard. to show this problem is in np, a polynomial time verifiable certificate could be the exact set $\{x_1 \dots x_k\}$ such that the formula comes out true. this can be verified in polytime of the input because each clause takes at most $O(1)$ to verify since we know there are only two arguments.

2. this problem is now in P. given there are n clauses and m different elements in the domain and a variables, we can break the query into a single variable queries. each element in the domain takes at most n attempts per query. in total it would take amn time which is in $\mathbb{P}$.